AWS VPC Peering Project

Name: Harshini Nirmalkumar
Batch: Batch 1, AWS Cloud Computing Internship (EWB Courses)
Project: Project 01, VPC Peering
Region: Asia Pacific (Sydney), ap-southeast-2, Availability Zone ap-southeast-2a
Completion date: 19 October 2026
GitHub repository: https://github.com/harshini346/aws-vpc-peering-lab

1. Objective

The goal was to connect two VPCs with VPC peering so that two private EC2 servers can exchange traffic over their private IPv4 addresses, while keeping both servers private. One public bastion host in VPC A is the only administration entry point. I built the lab, proved it failed before peering, proved it worked after peering, and ran a controlled failure to show the role of the security group rule.

The lab was built with AWS CLI scripts run in AWS CloudShell (see the GitHub repository) and verified from my laptop over SSH using ProxyJump through the bastion.

2. Architecture and resource inventory

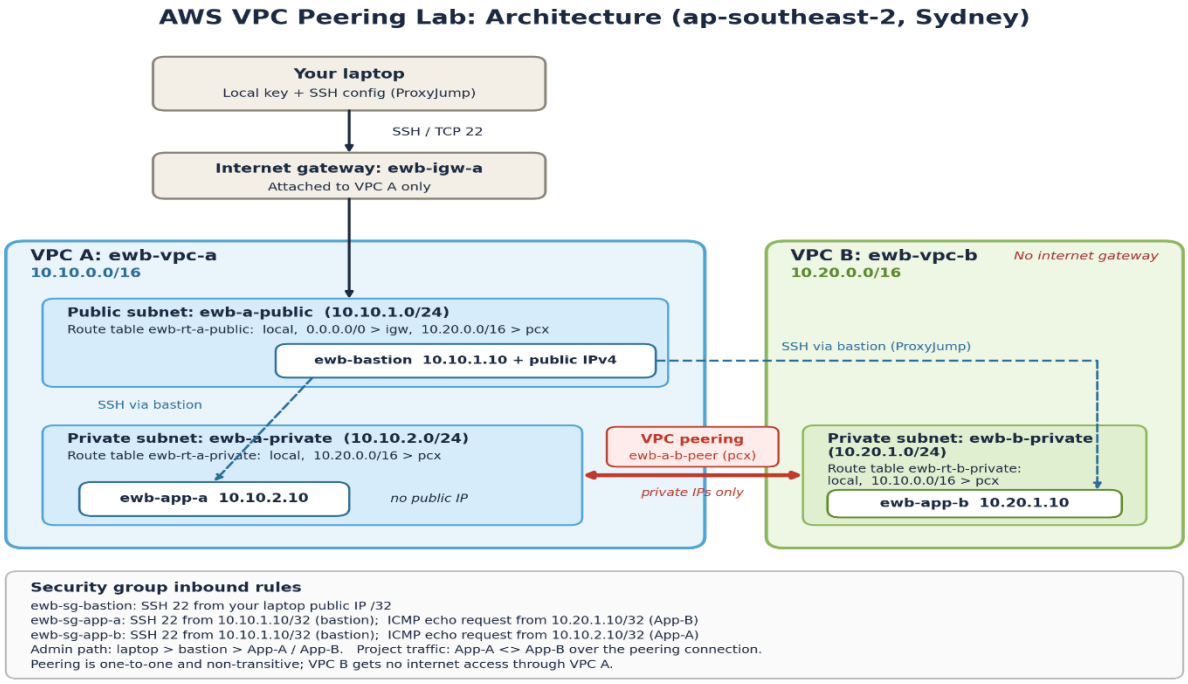


Fig 2.1: Architecture diagram of the lab: VPC A (10.10.0.0/16) with a public bastion and private App-A, VPC B (10.20.0.0/16) with private App-B, connected by VPC peering

Traffic paths

- Administration: laptop, then bastion, then App-A or App-B using SSH ProxyJump.
- Project traffic: App-A and App-B talk over private IPs through the peering connection.

- VPC B has no internet gateway. Peering does not give VPC B internet access through VPC A.

Resource	Name	CIDR / address	ID
VPC A	ewb-vpc-a	10.10.0.0/16	vpc-07dbd2ab24e89bfee
Public subnet A	ewb-a-public	10.10.1.0/24	subnet-058912c9289ec201c
Private subnet A	ewb-a-private	10.10.2.0/24	subnet-059d142816e1e2222
VPC B	ewb-vpc-b	10.20.0.0/16	vpc-0efd2c9c81942be99
Private subnet B	ewb-b-private	10.20.1.0/24	subnet-010a4045f66b7914e
Internet gateway	ewb-igw-a	attached to VPC A	igw-096079d859cfd8cdd
Peering connection	ewb-a-b-peer	requester VPC A, accepter VPC B	pcx-08c41d6b2ffbeb11e
Bastion	ewb-bastion	10.10.1.10 + public IPv4 3.106.122.133	i-073f0633fae84dc2d
App-A	ewb-app-a	10.10.2.10, no public IP	i-026276b5ced8293b3
App-B	ewb-app-b	10.20.1.10, no public IP	i-096a5c483ce4b6565

All resources were tagged Project = EWB-VPC-Peering and Student = Harshini. All instances are Amazon Linux 2023 on t3.micro with an 8 GB root volume set to delete on termination.

Why these CIDRs: 10.10.0.0/16 and 10.20.0.0/16 do not overlap, which is required for peering. Different subnet ranges alone would not fix overlapping VPC CIDRs.

3. Subnets, internet gateway and route tables

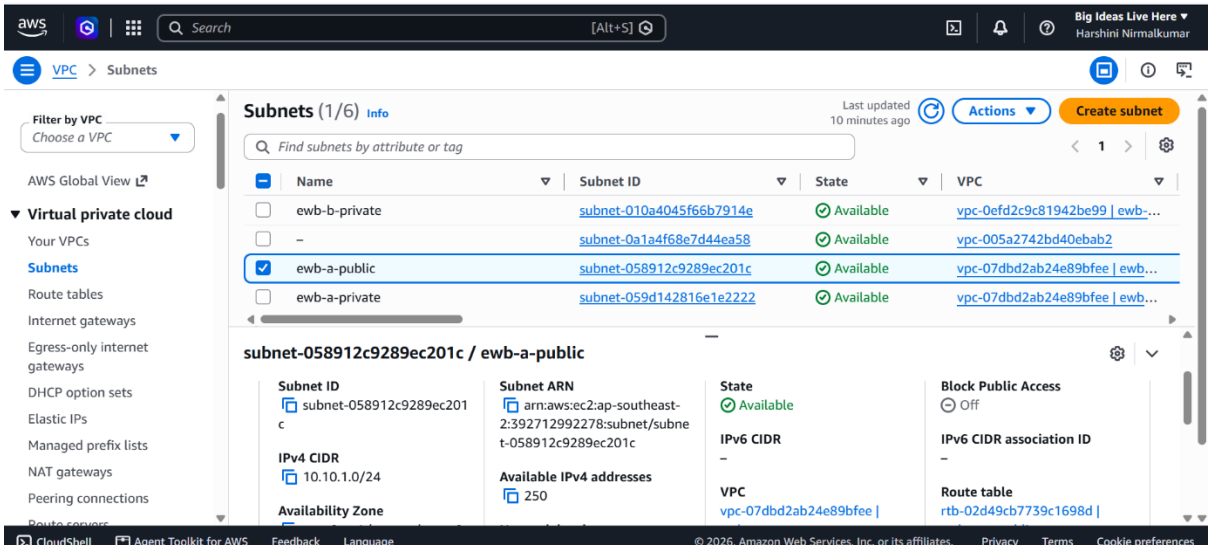


Fig 3.1: Subnet ewb-a-public (10.10.1.0/24) in VPC A, associated with route table rtb-02d49cb7739c1698d.

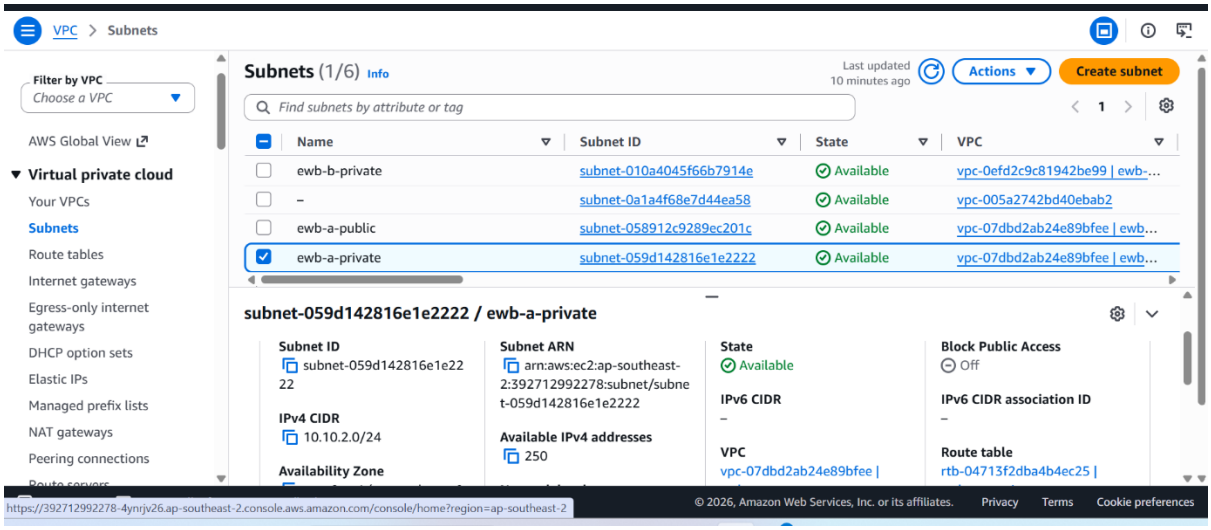


Fig 3.2: Subnet ewb-a-private (10.10.2.0/24) in VPC A, associated with route table rtb-04713f2dba4b4ec25.

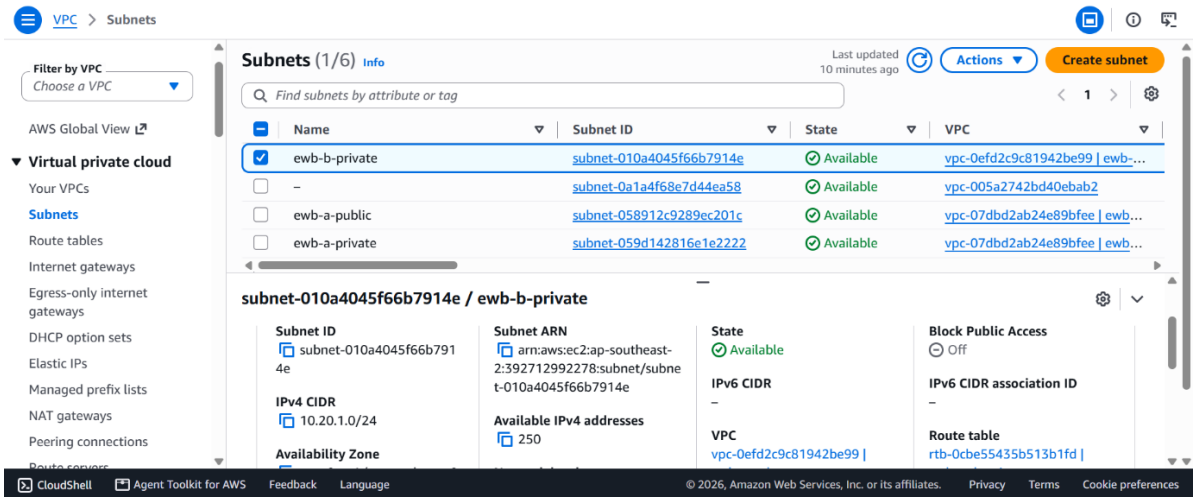


Fig 3.3: Subnet ewb-b-private (10.20.1.0/24) in VPC B, associated with route table rtb-0cbe55435b513b1fd.

Route table	Associated subnet	Association ID	Routes
ewb-rt-a-public (rtb-02d49cb7739c1698d)	ewb-a-public	rtbassoc-0a244bfefe346fe41	10.10.0.0/16 local; 0.0.0.0/0 to ewb-igw-a; 10.20.0.0/16 to peering
ewb-rt-a-private (rtb-04713f2dba4b4ec25)	ewb-a-private	rtbassoc-00c8c03f9607c645e	10.10.0.0/16 local; 10.20.0.0/16 to peering
ewb-rt-b-private (rtb-0cbe55435b513b1fd)	ewb-b-private	rtbassoc-0ec35e6bef6ab66d8	10.20.0.0/16 local; 10.10.0.0/16 to peering

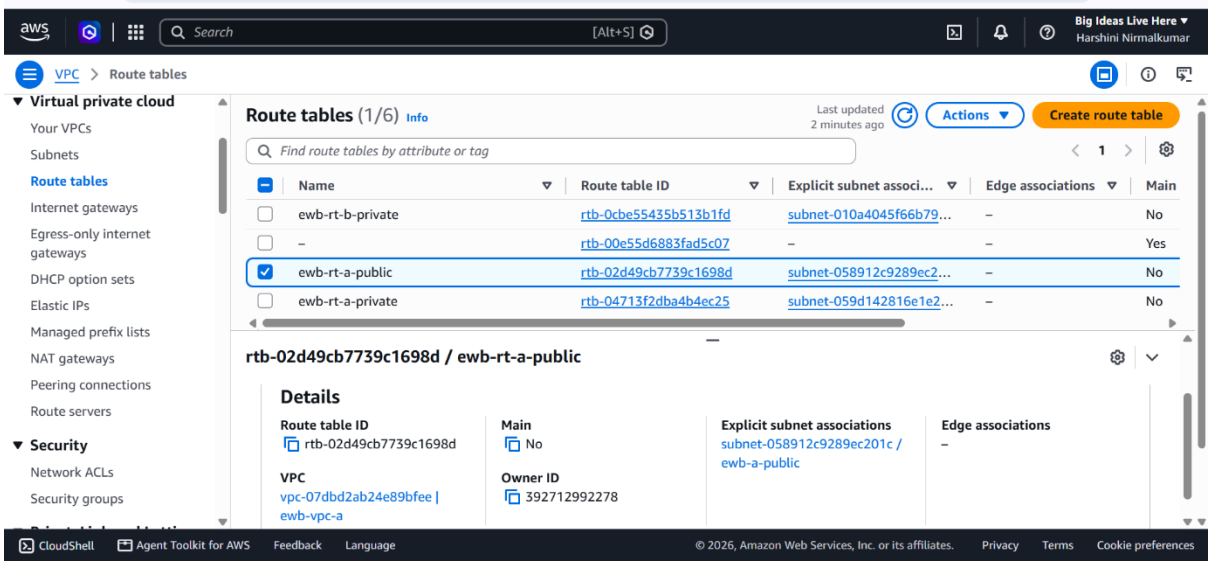


Fig 3.4: Route table ewb-rt-a-public, explicitly associated with subnet ewb-a-public.

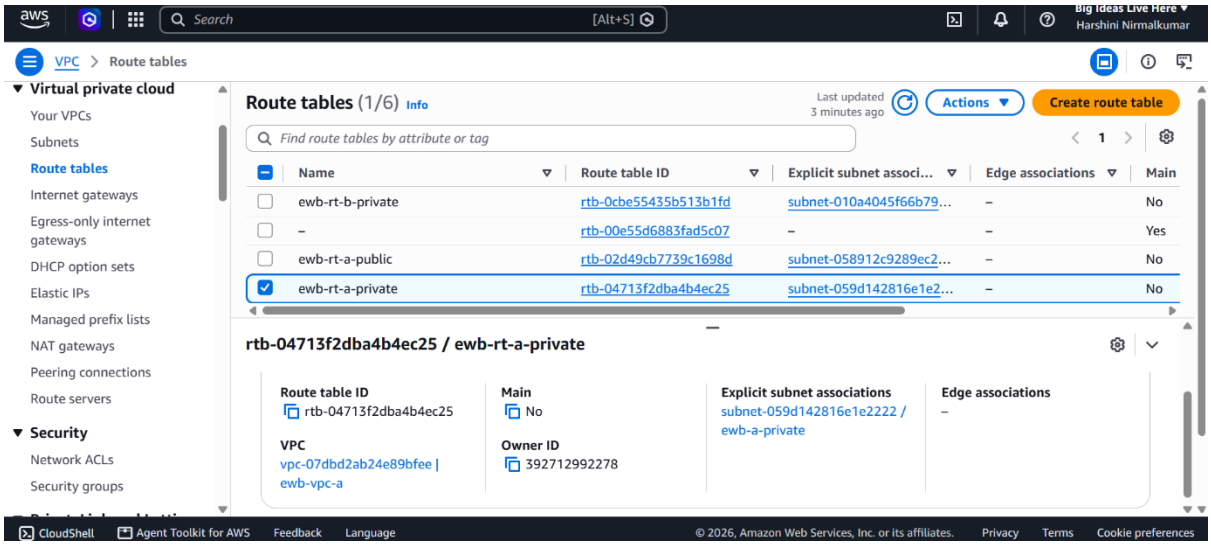


Fig 3.5: Route table ewb-rt-a-private, explicitly associated with subnet ewb-a-private.

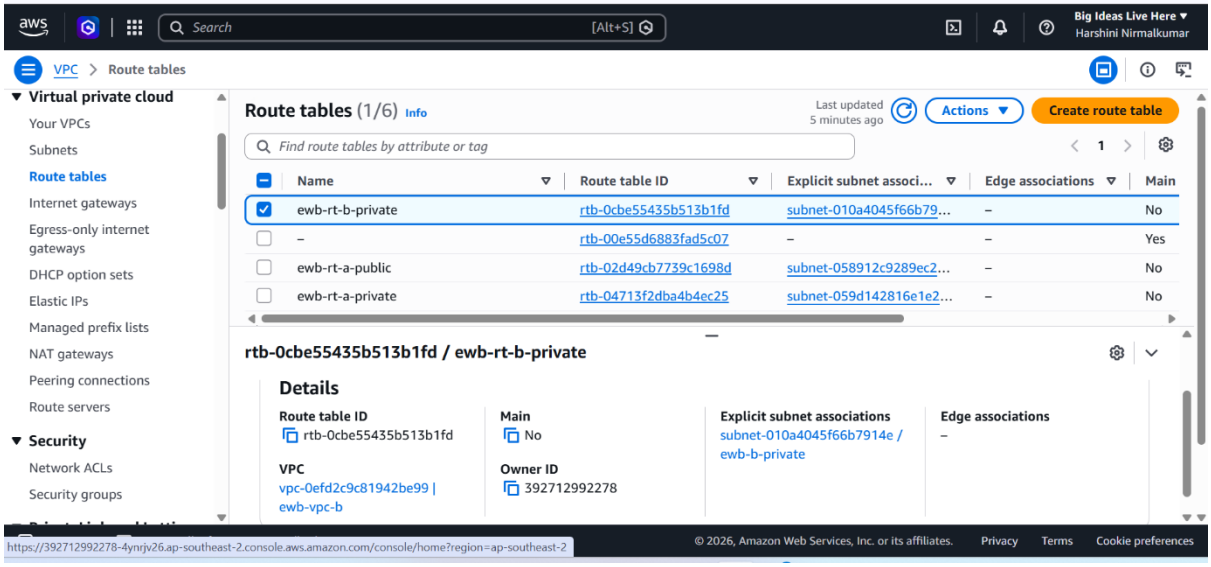


Fig 3.6: Route table ewb-rt-b-private, explicitly associated with subnet ewb-b-private.

The private route tables have no internet default route, and auto-assign public IPv4 is disabled on the private subnets. Only the bastion received a public IPv4 address.

4. Peering connection

The peering connection ewb-a-b-peer (pcx-08c41d6b2ffbeb11e) was requested from VPC A (requester) to VPC B (accepter) in the same account and Region. A request must be accepted even when both VPCs are in one account; I accepted it and waited for the status to become Active.

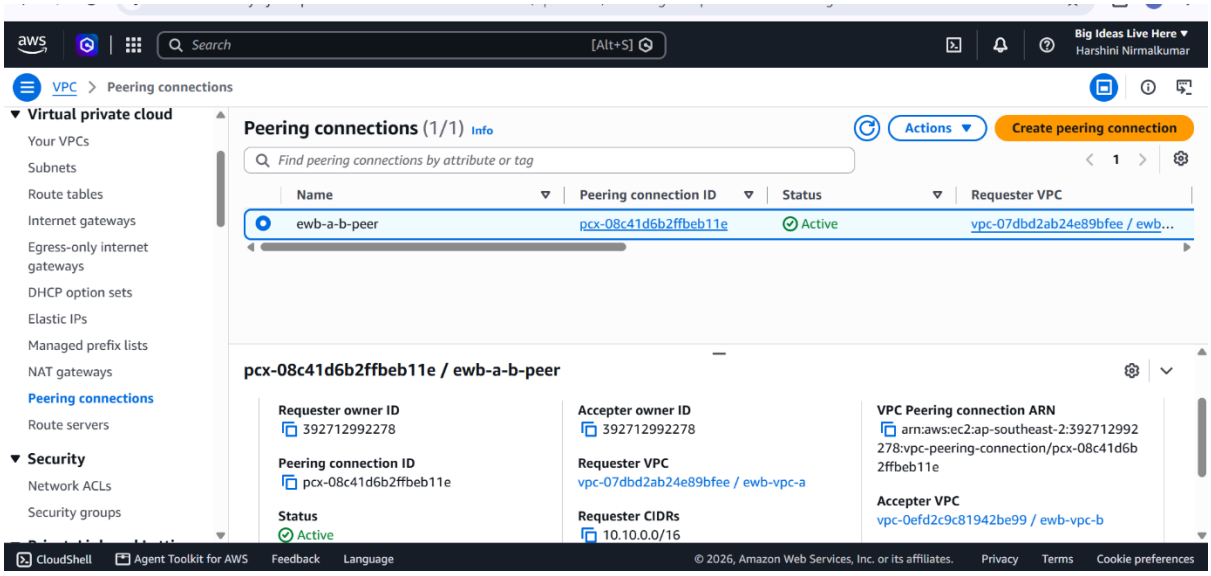


Fig 4.1: Peering connection ewb-a-b-peer (pcx-08c41d6b2ffbeb11e) in Active state, with VPC A as requester and VPC B as accepter.

5. Security group rules

Security group	Inbound rule	Source
ewb-sg-bastion (sg-0d753901aa3d1e6cf)	SSH, TCP 22	my laptop public IPv4 /32
ewb-sg-app-a (sg-03f37a9656894e809)	SSH, TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a	ICMP echo request (type 8, code 0)	10.20.1.10/32 (App-B)
ewb-sg-app-b (sg-0465c1abdcc1cac92)	SSH, TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b	ICMP echo request (type 8, code 0)	10.10.2.10/32 (App-A)

Outbound rules were left as the default allow-all, and network ACLs were the default allow-all. Security groups are stateful, so replies to permitted traffic are allowed automatically. SSH and ICMP were never opened to 0.0.0.0/0.

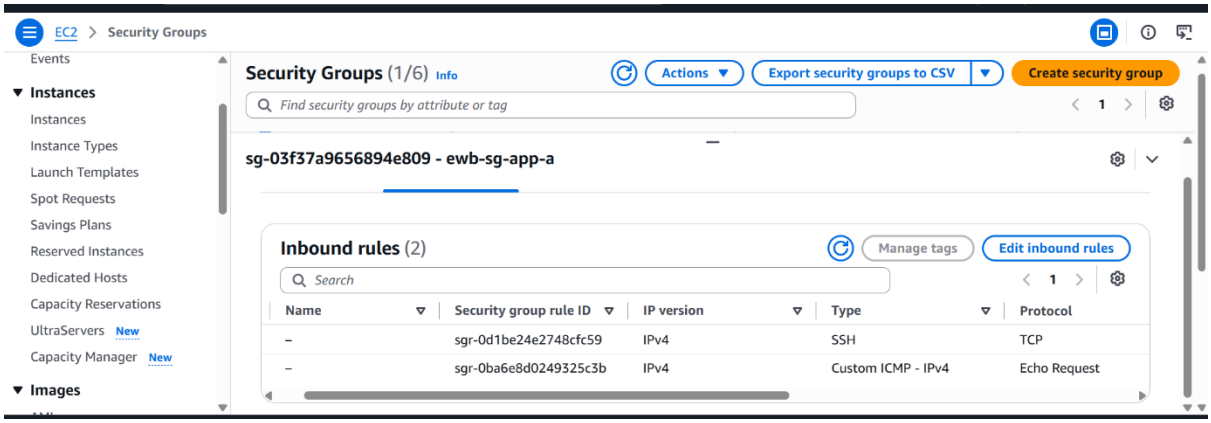


Fig 5.1: Inbound rules of ewb-sg-app-a: SSH from the bastion and ICMP echo request from App-B.

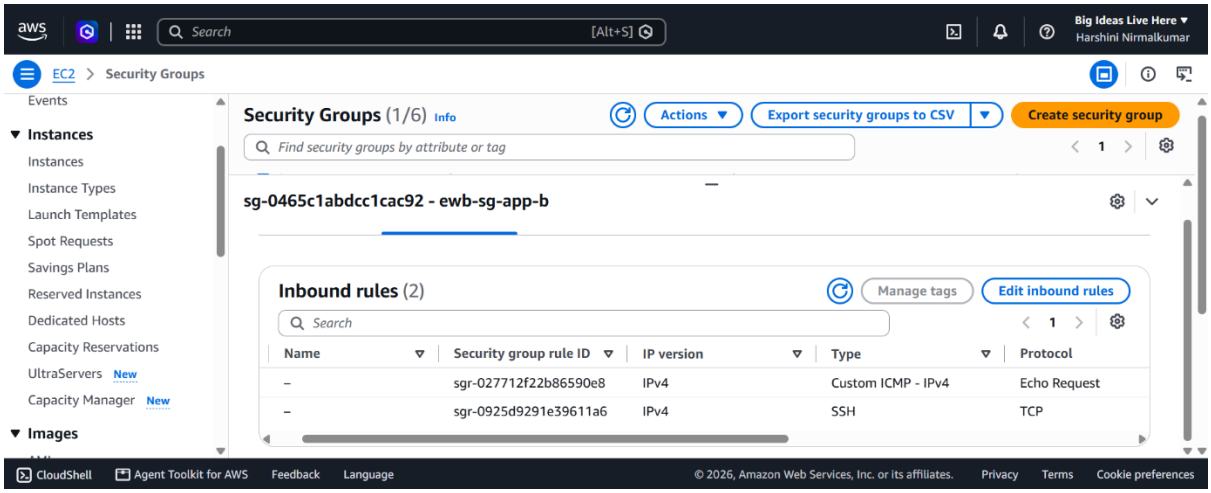


Fig 5.2: Inbound rules of ewb-sg-app-b: SSH from the bastion and ICMP echo request from App-A

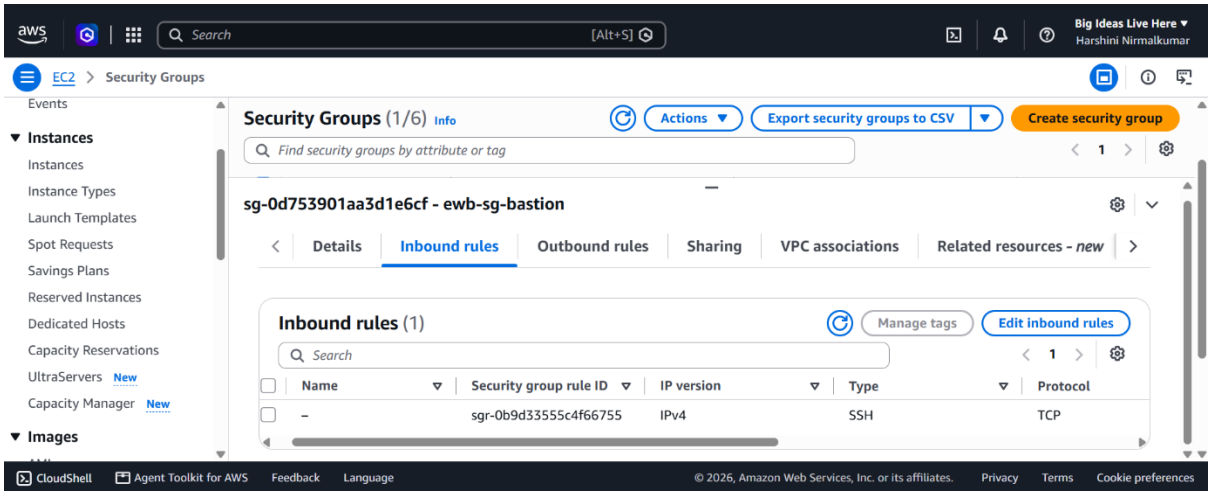


Fig 5.3: Inbound rule of ewb-sg-bastion: SSH (TCP 22) only from my laptop's public IP /32.

6. Private SSH access

I stored the key pair ewb-lab only on my laptop and never copied it to any instance. SSH uses ProxyJump through the bastion. I connected to the bastion first, then to App-A. After peering and routes were in place, I connected to App-B through the bastion as well.

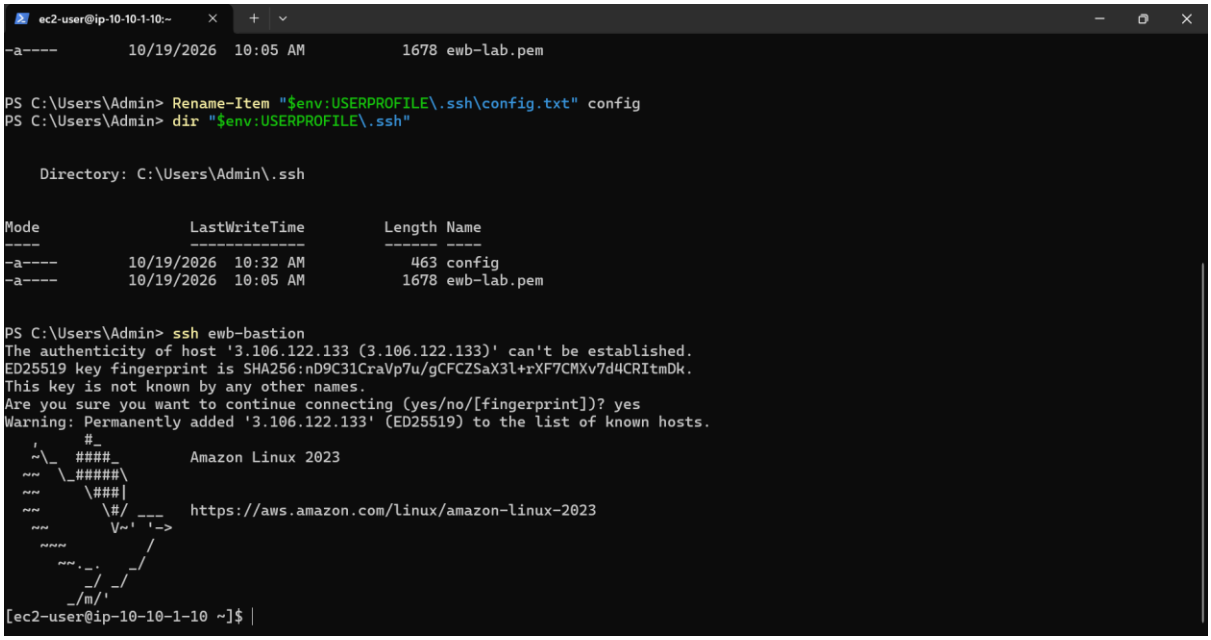


Fig 6.1: Laptop (PowerShell): SSH to ewb-bastion succeeds using the key and config, and the prompt shows the bastion's private IP 10.10.1.10.

7. Test results

Test	Where / command	Expected	Result
01 Baseline (before peering)	App-A: ping -c 4 10.20.1.10	No replies	4 transmitted, 0 received, 100% packet loss
02 Forward path (after routes)	App-A: ping -c 4 10.20.1.10	4 replies	4 transmitted, 4 received, 0% packet loss
03 Reverse path (after routes)	App-B: ping -c 4 10.10.2.10	4 replies	4 transmitted, 4 received, 0% packet loss
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Connected, prompt ip-10-20-1-10
05 Isolation	EC2 instance details	No public IPv4 on App-A and App-B	Public IPv4 field empty on both

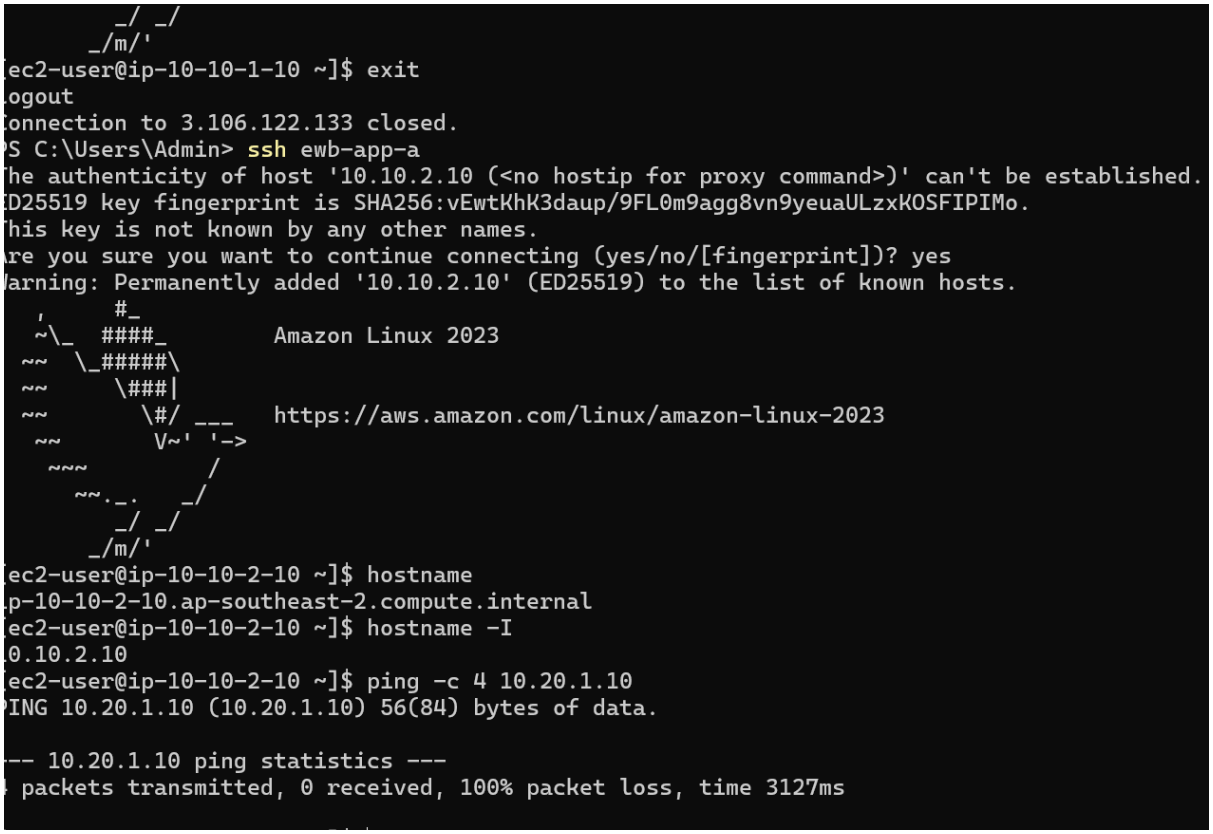


Fig 7.1 (baseline): App-A, before peering: ping -c 4 10.20.1.10 gets 0 received, 100% packet loss.



Fig 7.2: CloudShell: 02_peering.sh output. The peering is active and the routes to the other VPC are added in all three route tables.

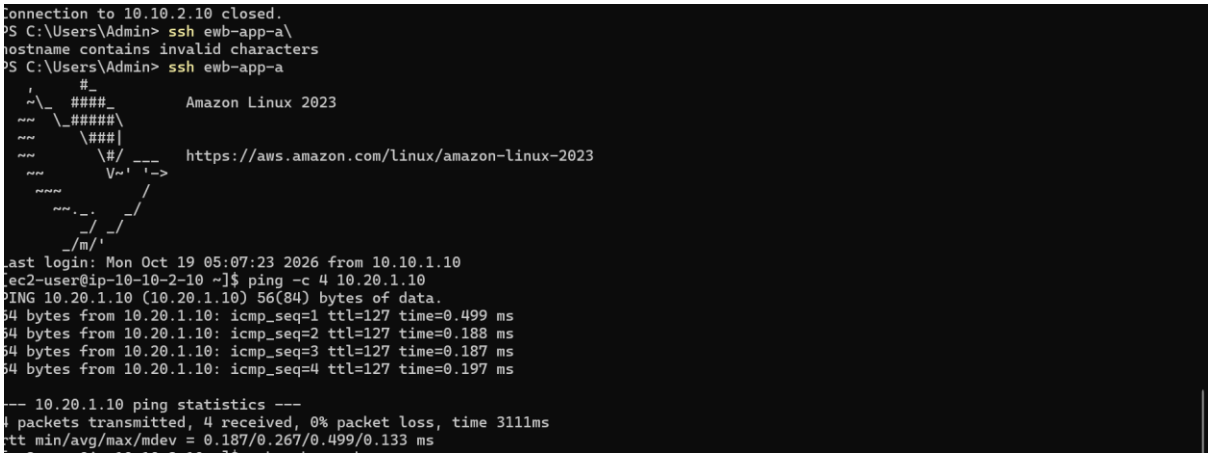


Fig 7.3 (forward path): App-A, after peering: ping -c 4 10.20.1.10 gets 4 received, 0% packet loss.

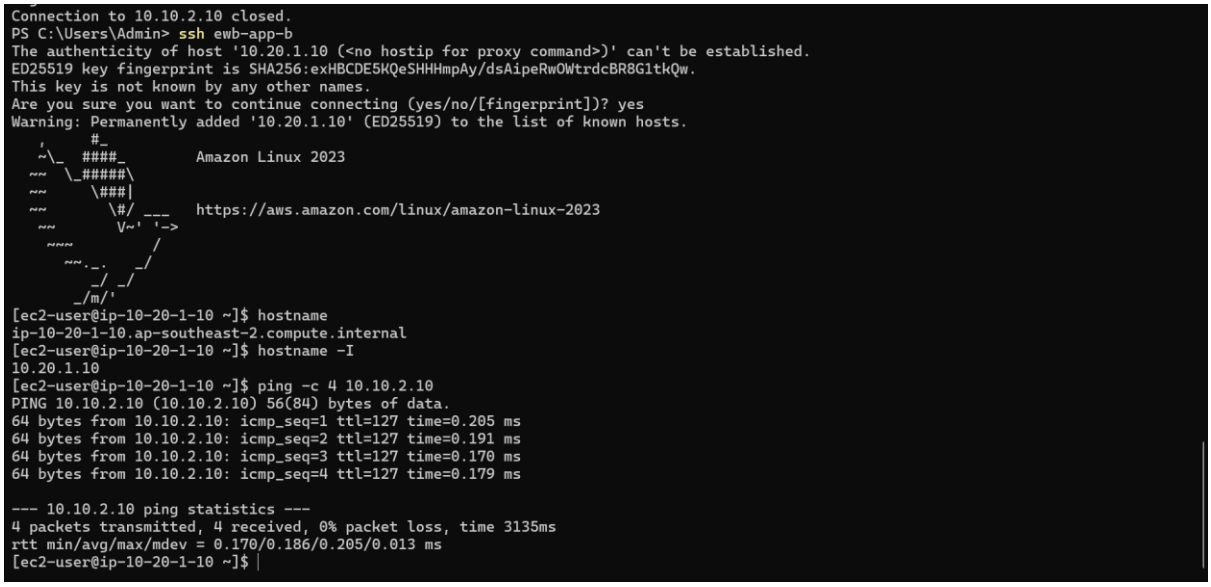


Fig 7.4 (reverse path and private SSH): Laptop to App-B through the bastion with ssh ewb-app-b, then on App-B ping -c 4 10.10.2.10 gets 4 received, 0% packet loss.

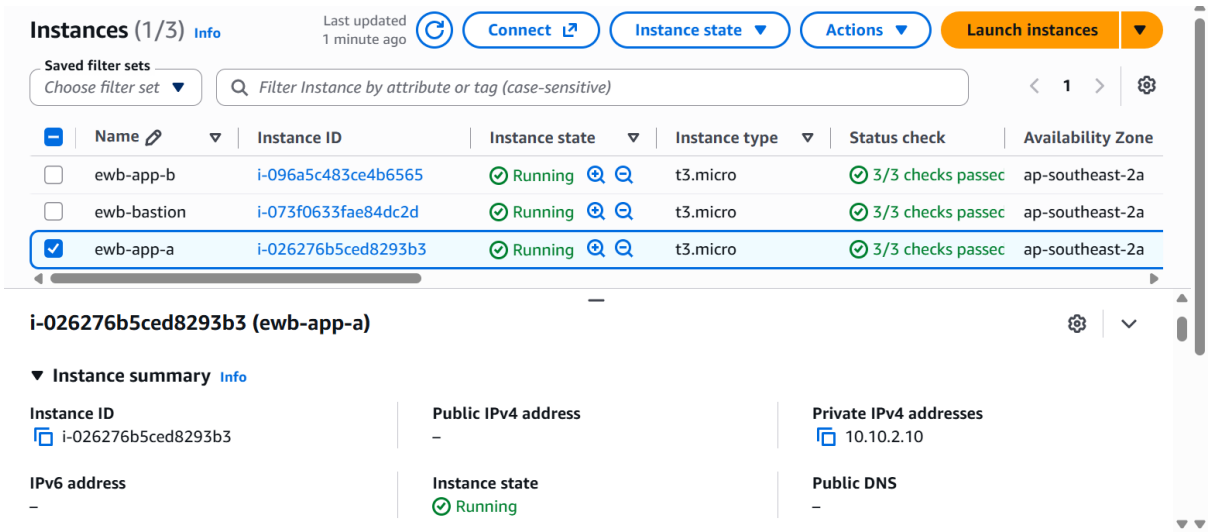


Fig 7.5: EC2 details of ewb-app-a: Public IPv4 address is empty; private IP is 10.10.2.10.

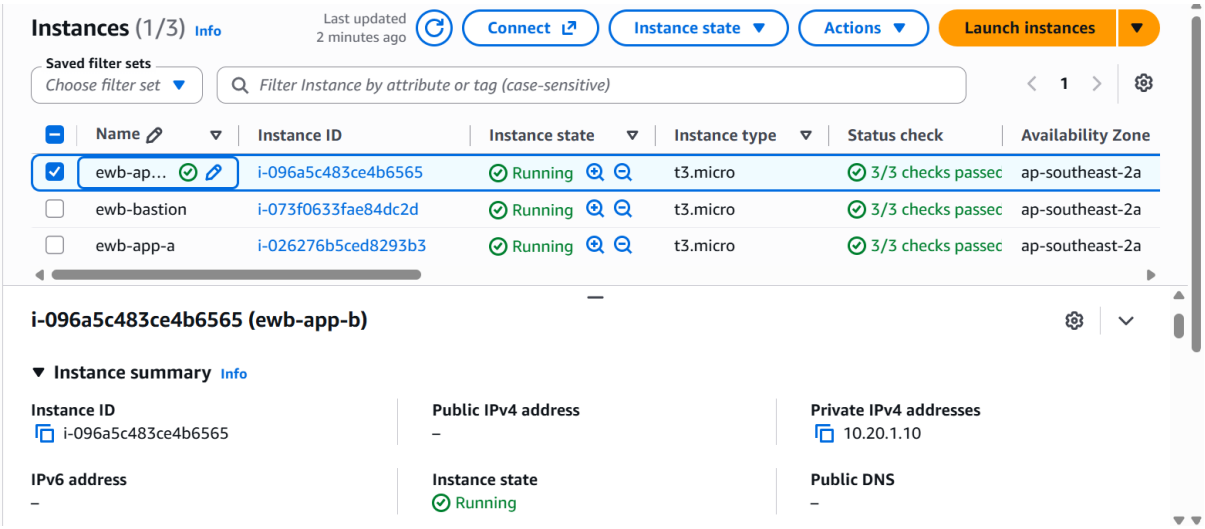


Fig 7.6: EC2 details of ewb-app-b: Public IPv4 address is empty; private IP is 10.20.1.10.

8. Controlled failure and recovery

I removed only the inbound ICMP echo request rule (source 10.10.2.10/32) from ewb-sg-app-b and kept the SSH rule. From App-A, ping -c 4 10.20.1.10 then returned 4 transmitted, 0 received, 100% packet loss. I restored the exact same rule and the ping returned 4 replies with 0% packet loss.

This shows that an Active peering connection and correct routes are not enough on their own: the destination security group must also allow the traffic. The route table decides whether a packet can reach the other VPC, and the security group decides whether the destination accepts it. The controlled failure test was run on a rebuilt copy of the same lab (same names, CIDRs and rules), so its resource IDs differ from the first build.



Fig 8.1: CloudShell: 04_controlled_failure.sh remove deletes only the ICMP echo rule (source 10.10.2.10/32) from ewb-sg-app-b.

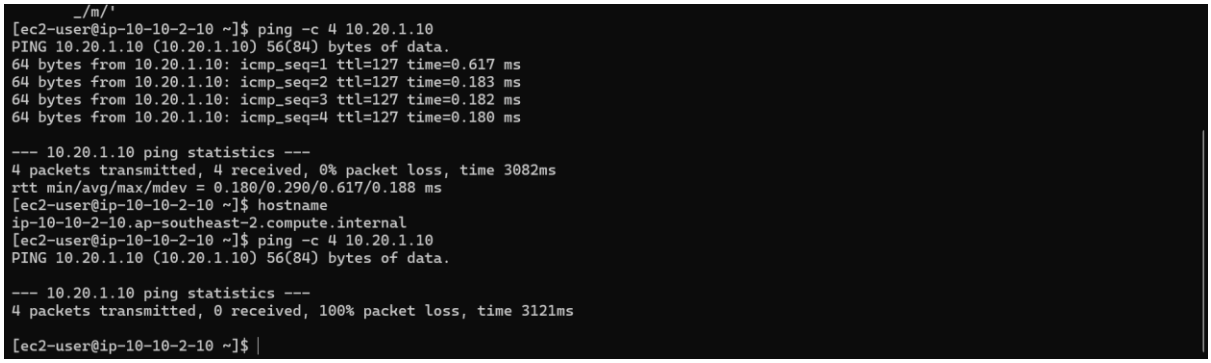


Fig 8.2: App-A with the rule removed: ping -c 4 10.20.1.10 gets 0 received, 100% packet loss.

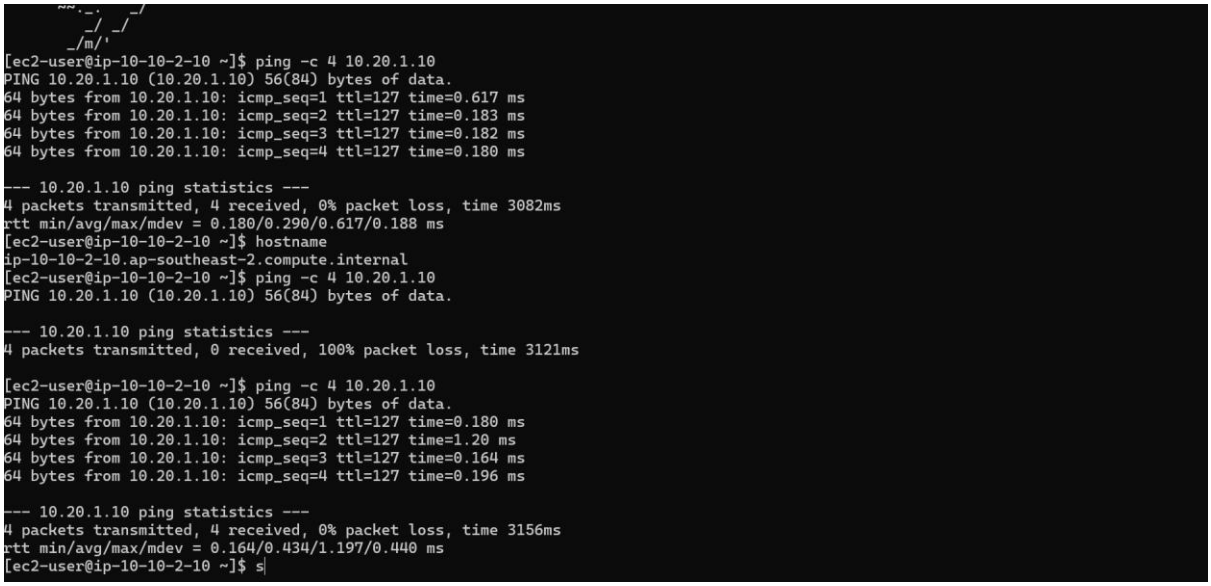


Fig 8.3: App-A after 04_controlled_failure.sh restore: the ping gets 4 received, 0% packet loss.

9. Forward path and return path

When App-A (10.10.2.10) pings App-B (10.20.1.10), the packet leaves App-A and ewb-rt-a-private matches 10.20.0.0/16 and sends it to the peering connection. It arrives in VPC B, where the security group ewb-sg-app-b allows the echo request from 10.10.2.10/32. App-B's reply is addressed to 10.10.2.10, and ewb-rt-b-private matches 10.10.0.0/16 and sends it back through the peering connection to App-A. If either route is missing, the ping or SSH session fails, because a one-way route is not enough.

10. Explanations

1. **Why is the return route needed?** The reply has to travel back to the other VPC. Without 10.10.0.0/16 pointing at the peering connection in VPC B's route table, App-B has no path to answer App-A.
2. **Why is the App-B SSH source the bastion private IP?** With ProxyJump, the bastion opens a separate SSH connection to App-B, so App-B sees the bastion's private IP 10.10.1.10 as the source, not my laptop.
3. **Why does Active peering not guarantee a successful ping?** Peering is only the link. Route tables must send traffic into it from both sides, and security groups (and network ACLs) must allow the traffic.
4. **Can VPC B use VPC A's internet gateway over peering?** No. Peering is one-to-one and non-transitive, and it does not provide internet access through the other VPC's gateway.

The bastion's public subnet also needs the 10.20.0.0/16 peering route, because the bastion itself opens the connection to App-B. The more specific 10.20.0.0/16 route is chosen over the internet default route.

11. Cleanup evidence

After saving all evidence, I terminated the three instances, deleted the peering routes and the peering connection, the security groups, the route table associations and route tables, the

subnets, the internet gateway and both VPCs, and deleted the ewb-lab key pair. I then checked EC2 instances, volumes and the VPC console for leftover resources.



Fig 11.1: CloudShell: 03_cleanup.sh output deleting the peering, security groups, route tables, subnets, internet gateway and VPCs.

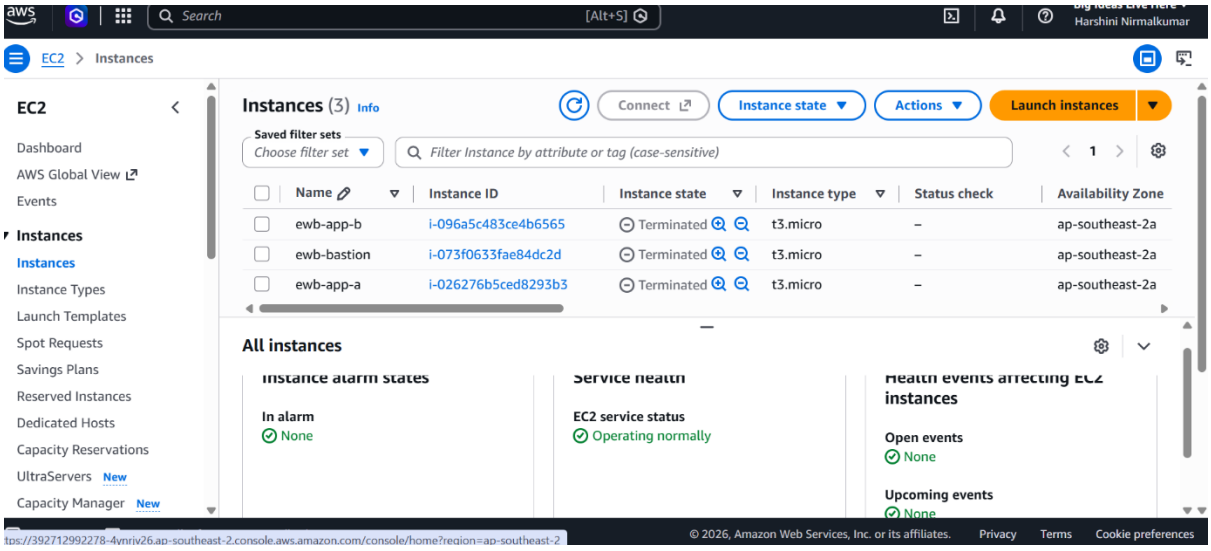


Fig 11.2: EC2 instances ewb-bastion, ewb-app-a and ewb-app-b all in Terminated state.

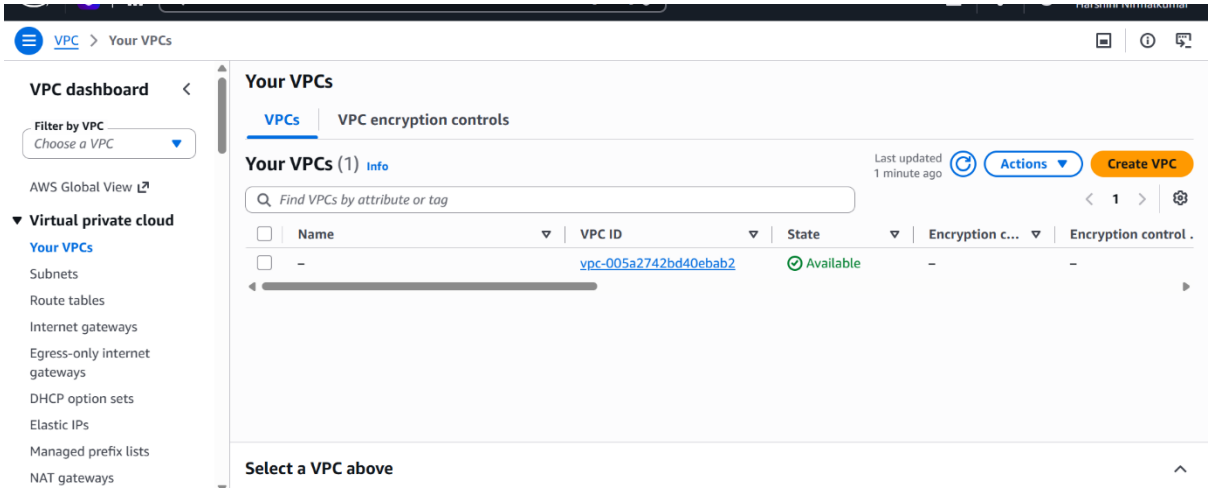


Fig 11.3: VPC console: only the default VPC remains, with no ewb VPCs.

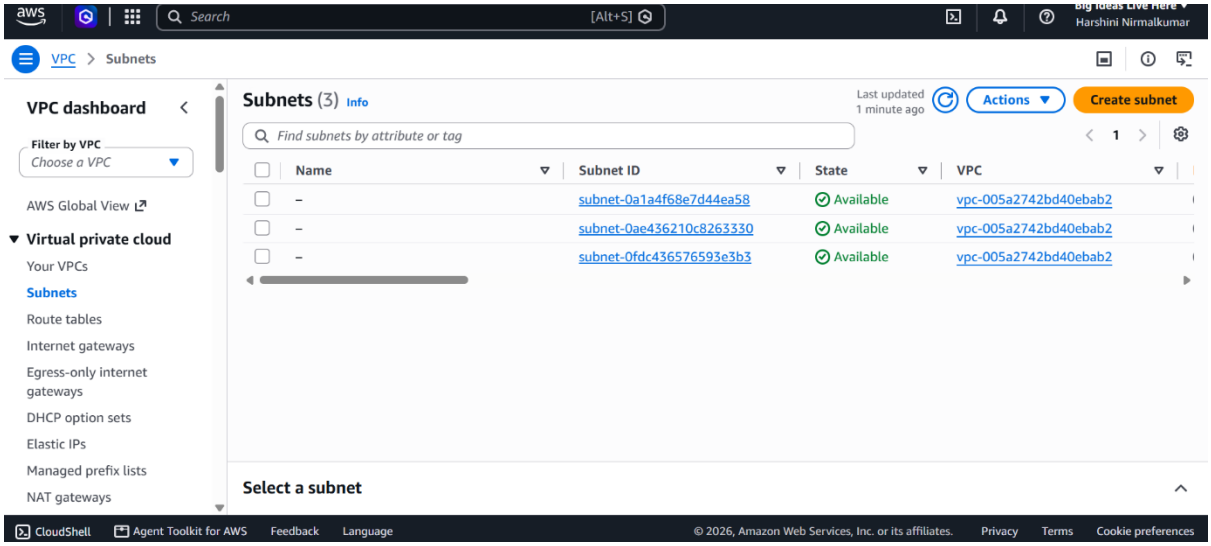


Fig 11.4: Subnets: only the default subnets remain.

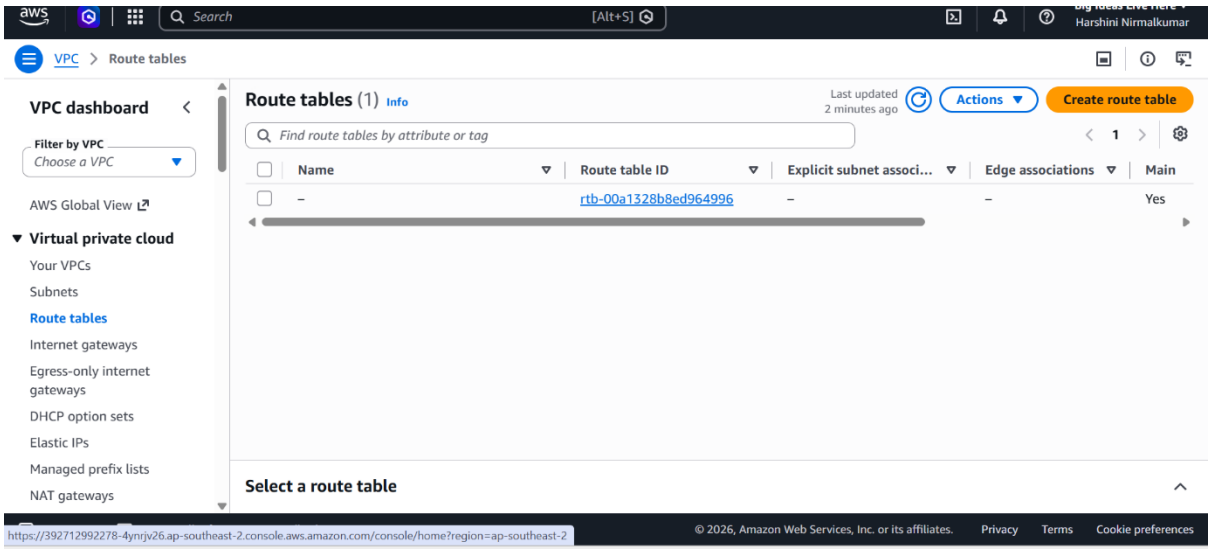


Fig 11.5: Route tables: only the default main route table remains.

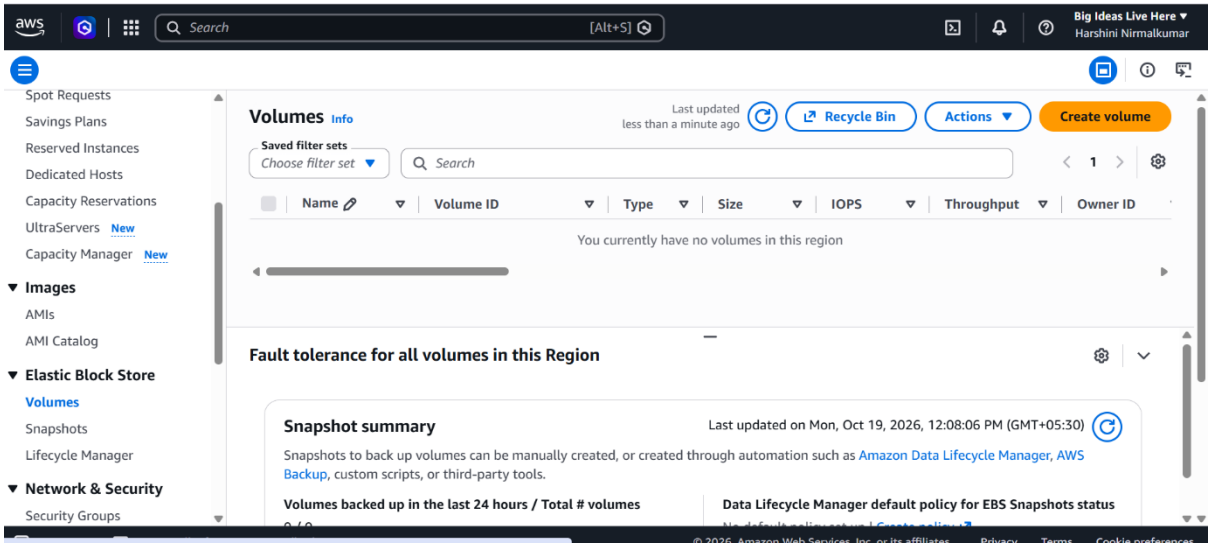


Fig 11.6: EC2 Volumes: no volumes left in the Region.

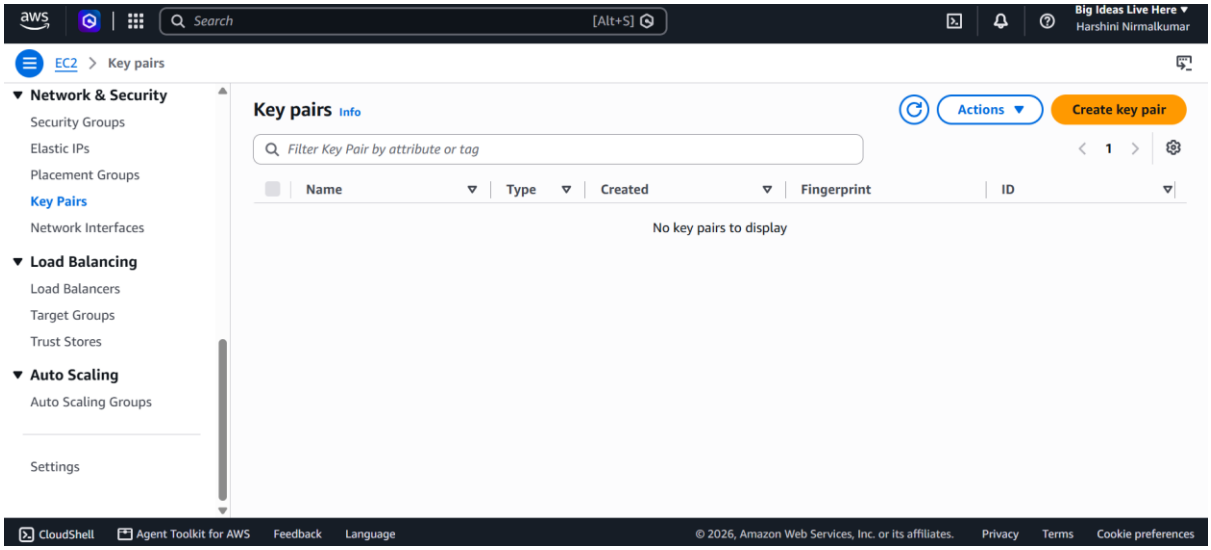


Fig 11.7 EC2 > Key Pairs after cleanup: the key pair ewb-lab is deleted and the list is empty.

12. What I learned

- A peering connection alone does nothing; routes on both sides and security group rules decide whether traffic flows.
- CIDR planning must happen first, because overlapping VPC ranges cannot be peered.
- A bastion host with SSH ProxyJump lets me administer private servers without copying keys or giving them public IPs.
- Testing before and after a change, and breaking something on purpose, proves that each rule is actually doing its job.
- Cleaning up promptly avoids unnecessary cost.